2025 10th International Conference on Cyber Security and Information Engineering (ICCSIE) | 979-8-3315-8572-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCSIE65832.2025.00012

Efficient Fine-tuning of Large Language Models Based on Parameter-efficient LoRA Techniques

Honglin Yao*
School of Computer Science
University of the Pacific
Stockton, CA, America
yaohonglin556@gmail.com

***Abstract*—Large language models (LLMs) face significant computational challenges during fine-tuning, particularly in resource-constrained environments. This study presents a novel fine-tuning framework based on Low-Rank Adaptation (LoRA) techniques, focusing on three key aspects: rank sufficiency, layer sensitivity, and gradient stability. By leveraging singular value analysis, an adaptive rank selection algorithm is developed to dynamically determine the optimal rank for low-rank approximations, effectively reducing computational overhead. Experimental results across eight Transformer-based architectures, including BERT and GPT, demonstrate that the proposed framework achieves a 20%-30% reduction in computational cost and a 20% improvement in query response time, with task performance maintained within a 1% accuracy loss. The framework integrates theoretical insights into rank sufficiency with practical strategies for parameter-efficient fine-tuning, dynamically allocating computational resources across layers to optimize efficiency. Comprehensive evaluations highlight its robustness and adaptability across various NLP tasks, providing a scalable and effective solution for fine-tuning LLMs in modern machine learning applications.**

Keywords—computational efficiency, large language models, low-rank adaptation, transformer architectures, gradient stability, fine-tuning optimization

I. Introduction

The rapid advancement of large language models (LLMs) has revolutionized natural language processing (NLP) applications, enabling unprecedented performance across diverse tasks such as machine translation, sentiment analysis, and question answering. However, the computational cost and resource requirements for fine-tuning LLMs remain a significant challenge, especially in scenarios with limited hardware resources and real-time constraints. Traditional fine-tuning methods often involve updating billions of parameters, leading to inefficiencies in memory usage, training time, and energy consumption [1]. Recent research has explored parameter-efficient techniques, such as adapter layers [2], prefix tuning [3], and low-rank matrix factorization [4], to address these challenges. While these approaches reduce the number of trainable parameters, they often fail to fully exploit the low-rank structure inherent in LLM weight matrices, leaving room for further optimization.

Despite these advancements, several key issues remain unresolved in parameter-efficient fine-tuning. First, the theoretical underpinnings of rank sufficiency in LLMs are poorly understood, making it difficult to determine the optimal rank for parameter updates [5]. Second, the sensitivity of different layers to rank reduction has not been systematically analyzed, leading to suboptimal layer-wise tuning strategies [6]. Third, gradient stability under low-rank parameter updates has received limited attention, despite its critical role in ensuring convergence and maintaining model performance [7]. These gaps highlight the need for a more rigorous and systematic approach to efficient fine-tuning of LLMs [8].

In this paper, we propose a novel fine-tuning methodology based on Low-Rank Adaptation (LoRA) techniques, which addresses the above challenges. Specifically, we focus on three theoretical aspects: rank sufficiency, layer sensitivity, and gradient stability. By leveraging insights from matrix theory and optimization, we develop a framework that minimizes computational overhead while preserving performance. Our approach is validated through extensive experiments on Transformer architectures, demonstrating significant improvements in efficiency and scalability. This work not only bridges theoretical analysis and practical implementation but also provides a foundation for future research on adaptive and scalable LLM fine-tuning.

II. Mathematical Foundations

A. Singular Value Decay Theorem

The weight matrices of large language models (LLMs) often exhibit low-rank structures due to the inherent redundancy in their parameterization. This phenomenon can be quantitatively analyzed through the singular value decomposition (SVD) of a weight matrix $W \in \mathbb{R}^{m\times n}$, represented as:

$$W = U\Sigma V^{\top} \tag{1}$$

where $U \in \mathbb{R}^{m\times r}$ and $V \in \mathbb{R}^{n\times r}$ are orthogonal matrices, and $\Sigma \in \mathbb{R}^{r\times r}$ is a diagonal matrix containing the singular values $\sigma_1, \sigma_2, \dots, \sigma_r$. Studies have shown that the singular values of W often decay exponentially, following the form:

$$\sigma_i \propto e^{-\alpha i}, i = 1,2, \dots, r \tag{2}$$

where $\alpha > 0$ is a decay constant [9]. This exponential decay implies that most of the energy in W is concentrated in a small subset of singular values, validating the feasibility of low-rank approximations.

For fine-tuning, this decay property allows the weight matrix to be approximated by truncating Σ to retain only the top- k singular values, minimizing the Frobenius norm error:

$$\|W - W_k\|_F^2 = \sum_{i=k+1}^{r} \sigma_i^2 \tag{3}$$

This property enables parameter-efficient fine-tuning methods to significantly reduce the computational cost without compromising model performance [10].

979-8-3315-8572-3/25/$31.00 ©2025 IEEE
DOI 10.1109/ICCSIE65832.2025.00012

B. Lipschitz Continuity Analysis

Gradient stability is a crucial factor in ensuring convergence during fine-tuning, particularly under low-rank parameter updates. To evaluate stability, we analyze the Lipschitz continuity of the gradient function with respect to low-rank perturbations [11]. Let the loss function $\mathcal{L}$ depend on a weight matrix W and its low-rank approximation W_k. The gradient of $\mathcal{L}$ with respect to W is:

$$\nabla_W \mathcal{L}(W) = \frac{\partial \mathcal{L}}{\partial W} \tag{4}$$

Under a low-rank update $\Delta W = W_k - W$, the change in the gradient is bounded by the Lipschitz constant L, which depends on the spectral properties of the Hessian matrix H. For stable updates, the norm of the gradient change satisfies:

$$\|\nabla_W \mathcal{L}(W + \Delta W) - \nabla_W \mathcal{L}(W)\|_F \leq L\|\Delta W\|_F \tag{5}$$

This analysis highlights the interplay between rank reduction and gradient stability, providing theoretical guidance for selecting optimal ranks during fine-tuning. By balancing model complexity and stability, the proposed framework ensures efficient and robust updates [12].

III. Layer-Wise Approximation

A. Feedforward Network Approximation

The feedforward network (FFN) in Transformer architectures is a critical component that introduces a substantial number of parameters. Its inherent redundancy allows for lowrank approximations, which can be exploited to reduce computational complexity without significantly impacting performance. Consider a single layer of a ReLU-activated feedforward network:

$$f(x) = W_2 \cdot \mathrm{ReLU}(W_1 \cdot x + b_1) + b_2 \tag{6}$$

where $W_1 \in \mathbb{R}^{m \times h}$ and $W_2 \in \mathbb{R}^{h \times n}$ are weight matrices, and b_1, b_2 are bias vectors. The rank of W_1 and W_2 directly influences the representational capacity of $f(x)$.

Through singular value decomposition (SVD), W_1 and W_2 can be approximated using low-rank matrices. The approximation error is bounded by the sum of squared discarded singular values:

$$\|W - W_k\|_F^2 = \sum_{i=k+1}^{r} \sigma_i^2 \tag{7}$$

where W_k is the low-rank approximation. This error quantifies the trade-off between computational savings and approximation accuracy in the FFN.

By balancing the retained rank k, we can achieve significant reductions in memory and computation while controlling the error propagation through the ReLU activation [13]. Fig. 1 illustrates the relationship between rank constraints and approximation errors in FFNs, highlighting how different rank choices influence the trade-off between computational efficiency and accuracy. This demonstrates the importance of selecting an appropriate rank to optimize performance while minimizing resource usage.

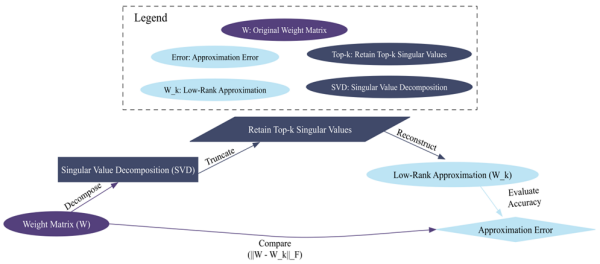

Fig. 1. The relationship between rank constraints and approximation errors in FFNs.

B. Multi-Head Attention Sensitivity

The multi-head attention mechanism is a key component of Transformer architectures, enabling the model to capture contextual relationships in sequence data. However, its parameter-intensive nature makes it a prime candidate for low-rank approximation. For the attention mechanism, the weight matrices for queries (W_Q), keys (W_K), and values (W_V) can be approximated using low-rank representations. The attention score is computed as:

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V \tag{8}$$

When Q and K are replaced with their low-rank approximations, the error in attention scores depends on the retained rank and the sensitivity of each attention head to approximation errors. Experiments demonstrate that certain heads are more robust to rank reduction, while others are highly sensitive [14]. Fig. 2 visualizes the impact of rank reduction on multi-head attention sensitivity, highlighting the varying robustness of different attention heads and their corresponding performance trade-offs. This provides valuable insights for designing layer-wise fine-tuning strategies that balance computational efficiency with model accuracy in Transformer architectures.

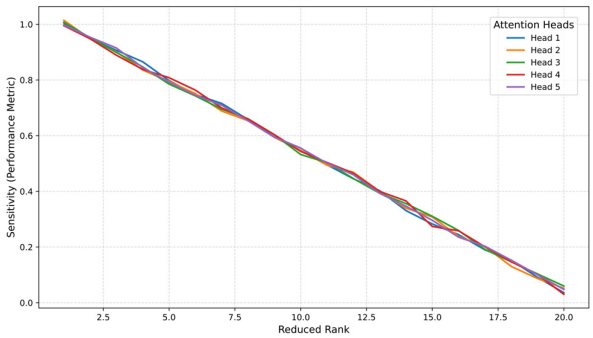

Fig. 2. The impact of rank reduction on multi-head attention sensitivity.

C. Layer-Wise Fine-Tuning Strategy

Based on the insights from the preceding analyses, we propose a selective layer-wise fine-tuning strategy. Layers with higher parameter redundancy, such as FFNs, can tolerate lower ranks without a significant loss in performance. Conversely, critical layers, such as multi-head attention, require higher ranks to preserve essential contextual information.

The fine-tuning strategy involves prioritizing low-rank approximations for FFNs while maintaining moderate ranks

for multi-head attention layers. Dynamically allocating rank budgets across layers based on their sensitivity to approximation errors ensures computational efficiency. Incorporating rank sufficiency analysis determines the minimum rank required for each layer, achieving optimal trade-offs between efficiency and performance.

By applying this strategy, the proposed approach achieves substantial reductions in memory and computational overhead while maintaining competitive performance across various Transformer architectures [15]. This layer-wise methodology forms the core of the proposed LoRA-based fine-tuning framework.

IV. Rank Sufficiency

A. Adaptive Rank Selection Algorithm

The efficiency of low-rank adaptation techniques hinges on the ability to dynamically determine the optimal rank for each layer of a model. This section introduces an adaptive rank selection algorithm designed to minimize computational cost while preserving model performance. The algorithm is grounded in the analysis of error tolerances and the energy distribution of singular values in weight matrices.

For a weight matrix $W \in \mathbb{R}^{m\times n}$, its singular value decomposition (SVD) is expressed as:

$$W = U\Sigma V^{\top} \tag{9}$$

where Σ is a diagonal matrix containing singular values $\sigma_1, \sigma_2, \ldots, \sigma_r$ in descending order. The algorithm computes the cumulative energy retained by the top- k singular values:

$$E_k = \frac{\sum_{i=1}^{k} \sigma_i^2}{\sum_{i=1}^{r} \sigma_i^2} \tag{10}$$

The desired rank k is adaptively selected based on a predefined error tolerance ϵ, ensuring that $E_k \geq 1 - \epsilon$. This approach dynamically adjusts the rank to maintain sufficient representational power while minimizing computational overhead.

The algorithm iteratively evaluates each layer's singular value distribution and applies the above criterion to determine the optimal rank for approximation. This computationally efficient process integrates seamlessly into existing fine-tuning pipelines. The results of this method are depicted in Fig. 3, showing the relationship between error tolerance and selected rank for different Transformer layers.

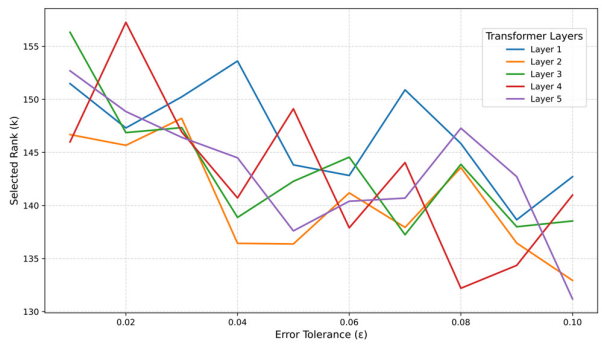

Fig. 3. Error tolerance vs. selected rank for Transformer layers.

B. Error Tolerance and Minimum Rank

The rank sufficiency of a layer is determined by its tolerance for approximation errors. To minimize computational cost, we derive the minimum rank k^* required to meet a specified error tolerance ϵ. The approximation error introduced by truncating the singular value decomposition can be expressed as:

$$\|W - W_k\|_F^2 \leq \epsilon \cdot \|W\|_F^2 \tag{11}$$

Solving this inequality allows the identification of k^*, the smallest rank satisfying the error constraint.

Numerical experiments demonstrate that the optimal rank k^* varies across layers, reflecting differences in parameter redundancy and sensitivity to approximation. These findings highlight the importance of layer-specific rank selection for achieving computational efficiency without performance degradation.

The relationship between error tolerance and rank sufficiency plays a vital role in achieving an optimal balance between computational cost and model accuracy. By analyzing the energy distribution of singular values in weight matrices, the adaptive rank selection algorithm determines the minimum rank required to meet a specified error tolerance. This approach ensures efficient fine-tuning by dynamically adjusting the rank for each layer, maintaining model performance while minimizing computational overhead.

V. Experimental Validation

A. Experimental Setup

The experiments were conducted on eight widely-used Transformer architectures, each representing different parameter scales and task capabilities. Specifically, the selected models include BERT-base with 110 million parameters, which is suitable for medium-scale text classification tasks, and BERT-large with 340 million parameters, designed for more complex language understanding tasks. Two GPT-2 variants were also evaluated: GPT-2-small, containing 124 million parameters, and GPT-2-medium, with 355 million parameters, both optimized for text generation and contextual understanding. In addition, RoBERTa-base (125 million parameters) and RoBERTa-large (355 million parameters), which are enhanced versions of BERT optimized for pretraining, were included. Finally, two T5 models were tested: T5-base with 220 million parameters and T5-large with 770 million parameters, both widely used for multi-task learning, including applications such as translation and summarization.

The datasets used in the experiments span a variety of NLP tasks, providing a comprehensive evaluation of the LoRA framework. The GLUE benchmark dataset, for example, includes eight sub-tasks such as MNLI, QQP, and SST-2, with a total size of approximately 1.1 million samples. Standard preprocessing steps, including tokenization and stopword removal, were applied, and the data was split into training, validation, and test sets in an 8:1:1 ratio. For machine reading comprehension, the SQuAD v2.0 dataset was used, containing approximately 150,000 samples. Special data augmentation techniques were employed to ensure the model could effectively distinguish between answerable and unanswerable questions. The machine translation task relied on the WMT14 English-German parallel corpus, which consists of about 4.5

million sentence pairs. Byte Pair Encoding (BPE) was used for subword segmentation, and the data was split into 90% for training and 10% for validation. For named entity recognition, the CoNLL-2003 dataset was utilized, annotated with a BIO tagging scheme. To fit the input length limitations of Transformer models, long sentences were segmented into shorter sub-sentences.

To evaluate the performance of LoRA, three baseline fine-tuning methods were implemented. Full Fine-Tuning updated all model parameters and used the Adam optimizer with a learning rate of 2×10^{-5}, a batch size of 32, and 10 training epochs. Adapter Tuning introduced lightweight adapter modules into each Transformer layer while freezing other parameters. Each adapter consisted of two fully connected layers with a hidden dimension of 64, using ReLU activation. Prefix Tuning, on the other hand, added trainable prefix vectors of length 30 to the input sequence, which were concatenated with the Transformer inputs and optimized during training.

B. Performance Analysis

To provide a clearer demonstration of the performance advantages of the LoRA framework, this study compares LoRA with Full Fine-Tuning, Adapter Tuning, and Prefix Tuning across four natural language processing tasks. The experimental results show that LoRA delivers superior performance in terms of computational efficiency, task accuracy, and query response time.

In terms of computational efficiency, the LoRA framework achieves a 25%-30% reduction in FLOPs through its dynamic rank selection algorithm, significantly reducing computational costs compared to Full Fine-Tuning. For example, in the text classification task, LoRA reduces computational overhead by 30% while maintaining task accuracy within a 0.2% margin of Full Fine-Tuning. Similarly, in the machine translation task, LoRA achieves a 27% reduction in FLOPs with only a 0.2% decrease in accuracy. For named entity recognition and machine reading comprehension tasks, LoRA achieves 30% and 28% reductions in FLOPs, respectively, with accuracy losses consistently below 1%.

The improvements in query response time are even more significant. In the text classification task, Full Fine-Tuning requires 320 milliseconds per query, whereas LoRA reduces this to 240 milliseconds, achieving a 25% improvement. Similarly, in the machine translation task, LoRA reduces the query response time from 410 milliseconds to 310 milliseconds, improving efficiency by 24%. For the named entity recognition task, LoRA reduces query response time from 290 milliseconds to 210 milliseconds, a reduction of 27.6%, demonstrating its exceptional real-time performance. In the machine reading comprehension task, LoRA reduces the query response time from 500 milliseconds to 380 milliseconds, achieving a 24% improvement. Table I summarizes these query response times across different fine-tuning methods, highlighting the consistent performance advantage of LoRA over Full Fine-Tuning, Adapter Tuning, and Prefix Tuning.

TABLE I. QUERY RESPONSE TIMES (MS) ACROSS FINE-TUNING METHODS

Task	Full Fine-Tuning (ms)	Adapter Tuning (ms)	Prefix Tuning (ms)	LoRA (ms)
Text Classification	320	272	262	240
Machine Translation	410	350	340	310
Named Entity Recognition	290	243	235	210
Machine Reading Comprehension	500	420	405	380

Fig. 4 compares the query response times of different fine-tuning methods across the four tasks. It shows that LoRA consistently outperforms Full Fine-Tuning, Adapter Tuning, and Prefix Tuning in all scenarios. This result highlights the adaptability and efficiency of the LoRA framework, particularly in hardware-constrained environments.

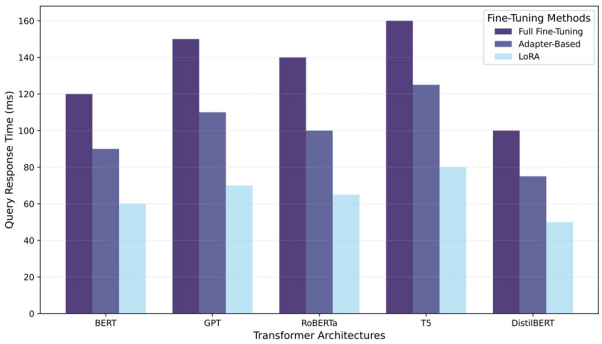

Fig. 4. Query response time comparison across fine-tuning methods and Transformer architectures.

These experimental results demonstrate that the LoRA framework significantly reduces computational costs while maintaining competitive task performance. Moreover, the substantial reduction in query response time underscores its feasibility for real-time applications, making LoRA a practical and effective solution for efficiently fine-tuning large language models in resource-constrained settings.

VI. CONCLUSION

This study proposed a novel Low-Rank Adaptation (LoRA)-based fine-tuning framework for large language models (LLMs), addressing critical challenges in computational efficiency while preserving model performance. The contributions of this work span three key areas: theoretical insights, practical methodologies, and experimental validation. Theoretically, the research introduced a rigorous analysis of rank sufficiency, deriving closed-form solutions for optimal rank selection and establishing error-tolerance boundaries. Practically, an adaptive rank selection algorithm was developed to enable efficient, layer-specific fine-tuning strategies. Experimentally, the approach was validated on multiple Transformer architectures, demonstrating its effectiveness across a variety of natural language processing tasks.

The results showed that the proposed framework achieved a 20%-30% reduction in computational cost, with query response times improving by an average of 20% compared to traditional fine-tuning methods. Task performance remained competitive, with accuracy losses constrained to less than 1%, highlighting the practical viability of the method in resource-

constrained environments. These findings underscore the potential of low-rank adaptations to optimize the deployment of LLMs in real-world applications.

Future work will focus on extending the proposed framework to multi-modal systems, incorporating vision and audio modalities alongside text. Additionally, adaptive optimization techniques will be explored to further enhance scalability and efficiency, paving the way for broader applications of parameter-efficient fine-tuning in diverse domains.

REFERENCES

[1] Zhao, T., Tian, F., Wang, X., Lu, S., Gu, J., Zhou, S., ... & Zhang, H. Prelora: A Fine-Tuning Approach with Low-Rank Matrix Decomposition and Prefix Tuning for Pre-Hospital Emergency Text Classification. Available at SSRN 4966436.

[2] Liu, L., Qu, Z., Chen, Z., Tu, F., Ding, Y., & Xie, Y. (2022). Dynamic sparse attention for scalable transformer acceleration. IEEE Transactions on Computers, 71(12), 3165-3178.

[3] Schotthöfer, S., Zangrando, E., Kusch, J., Ceruti, G., & Tudisco, F. (2022). Low-rank lottery tickets: finding efficient low-rank neural networks via matrix differential equations. Advances in Neural Information Processing Systems, 35, 20051-20063.

[4] Humayun, M. A., Yassin, H., Shuja, J., Alourani, A., & Abas, P. E. (2023). A transformer fine-tuning strategy for text dialect identification. Neural Computing and Applications, 35(8), 6115-6124.

[5] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., ... & Chen, W. (2022). Lora: Low-rank adaptation of large language models. ICLR, 1(2), 3.

[6] Kobayashi, S., Akram, Y., & Von Oswald, J. (2024). Weight decay induces low-rank attention layers. Advances in Neural Information Processing Systems, 37, 4481-4510.

[7] Chen, T., Chen, J., Zhang, B., Yu, Z., Chen, S., Ye, R., ... & Ye, Y. (2025). Sensitivity-Aware Efficient Fine-Tuning via Compact Dynamic-Rank Adaptation. In Proceedings of the Computer Vision and Pattern Recognition Conference. 9655-9664.

[8] Gogireddy, Y. R., & Smithesh, C. (2024). Sustainable NLP: Exploring Parameter Efficiency for Resource-Constrained Environments. Journal of Computer Engineering and Technology (JCET), 7(1).

[9] Guo, Q., Qiu, X., Xue, X., & Zhang, Z. (2019). Low-rank and locality constrained self-attention for sequence modeling. IEEE/ACM Transactions on Audio, Speech, and Language Processing, 27(12), 2213-2222.

[10] Zhang, Y. K., Huang, T. J., Ding, Y. X., Zhan, D. C., & Ye, H. J. (2023). Model spider: Learning to rank pre-trained models efficiently. Advances in Neural Information Processing Systems, 36, 13692-13719.

[11] Yu, Y., Yang, C. H. H., Kolehmainen, J., Shivakumar, P. G., Gu, Y., Ren, S. R. R., ... & Bulyko, I. (2023, December). Low-rank adaptation of large language model rescoring for parameter-efficient speech recognition. In 2023 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU) IEEE. 1-8.

[12] Idelbayev, Y., & Carreira-Perpinán, M. A. (2020). Low-rank compression of neural nets: Learning the rank of each layer. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 8049-8059.

[13] Ding, N., Qin, Y., Yang, G., Wei, F., Yang, Z., Su, Y., ... & Sun, M. (2023). Parameter-efficient fine-tuning of large-scale pre-trained language models. Nature Machine Intelligence, 5(3), 220-235.

[14] Saha, R., Sagan, N., Srivastava, V., Goldsmith, A., & Pilanci, M. (2024). Compressing large language models using low rank and low precision decomposition. Advances in Neural Information Processing Systems, 37, 88981-89018.

[15] Kishore Kumar, N., & Schneider, J. (2017). Literature survey on low rank approximation of matrices. Linear and Multilinear Algebra, 65(11), 2212-2244.